

Lab 3. Research assistant over MCP

A question in. A cited brief out. Same graph, different object.

25 minutes. Fill `loop.py`. Two functions. Nothing else.

Work from `labs/lab3_research`.

Same graph Orchestrator plus Doer plus Judge



What you will build

Two function bodies:

```
def plan_questions(question: str) -> list[str]: ...  
def check_brief(body: str, sources: list[str]) -> brief.BriefScore: ...
```

Already shipped: `brief.py` (the judge). Do not reimplement grounding.

Final outcome. Three checkable sub-questions. Four arithmetic rows on the brief. No model in the judge.

Why this lab exists

A code loop stops when tests go green. A research loop has no equivalent. "Keep searching until confident" is not a stop condition.

Tool output is untrusted input. Search results are a document the internet wrote, not a system prompt.

Saturday uses the fixture backend. No signup form.

Learning objectives

- Implement `plan_questions` as three checkable sub-questions
- Implement `check_brief` as `return brief.check(body, sources)`
- Configure a tool boundary: search in, merge denied
- Validate `has_sources`, `grounded`, `cited`, `style`
- Troubleshoot the stall of reaching for a model judge

Spillwave Solutions | spillwave.com

Starting architecture

```
orchestrator  owns budget + exits. Writes nothing. Sees summaries.  
+-- researcher  search tools only. Isolated context. No write.  
+-- writer      brief.md + work/research/** only.  
+-- judge       brief.check. Citations are arithmetic.
```

Allowed: search, write this loop's own output folder.

Denied: merge, deploy, ticket state, CRM edits.

Prerequisites

```
cd labs/lab3_research
claude -p "$(cat prompts/claude-code.md)"
```

Worth reading: `brief.py`, `MCP.md`.

`.mcp.json` at the repo root ships `context7`. Perplexity is optional. Fixture when the room has no wifi.

`.claude/settings.json` denies writes to `loops/`, `scripts/`, and CRM tests. There is no `loops/` package.

Spillwave Solutions | spillwave.com

The stub

```
def plan_questions(question: str) -> list[str]:  
    raise NotImplementedError("fill me in")  
  
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:  
    raise NotImplementedError("fill me in")
```

Fill only `loop.py`. Do not edit `brief.py`. Do not edit `solutions/`.

Spillwave Solutions | spillwave.com

`plan_questions`. **Three strings is enough**

A plan step you cannot check is a wish. The common stall is over-designing this function. This is Graph Engineering at lab scale: three checkable nodes, not a wish list.

```
def plan_questions(question: str) -> list[str]:  
    return [  
        question,  
        f"{question} common mistake",  
        f"{question} how to verify",  
    ]
```

That is the body in `solutions/sol3_research_deep_agents/researcher.py`. Swapping in a model planner is the stretch. Downstream checks do not change.

check_brief. **Arithmetic. No model**

```
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    return brief.check(body, sources)
```

ROW		PASS WHEN
has_sources	bool(sources)	
grounded	every [n] is in 1..len(sources)	
cited	every claim paragraph has a [n]	
style	zero em dashes outside code spans	

A confident sentence nobody can trace is the failure that matters.

Why style is a row

House style forbids em dashes. A model will argue. Python will not.

`brief.strip_em_dashes` stashes code spans, then comma-heavy lines become `:`, light lines become `;`.

Spillwave Solutions | spillwave.com

Commands. Live first.

`task loop:research` is gone with `loops/`.

Saturday self-check:

```
task test    # import loop; print('ok')
```

Runnable filled loop, after class or as the instructor demo:

```
cd ../../solutions/sol3_research_deep_agents
python3 loop.py --question "sqlalchemy nullable datetime column" --backend fixture
```

The question is boring on purpose. It matches `fixtures/research.json`.

Expected fixture result

```
PASS  has_sources      3 sources retrieved
PASS  grounded         every citation resolves
PASS  cited            every paragraph cites a source
PASS  style            0 em dashes

backend: fixture      budget: $0.00 / $0.20 (soft $0.10), 3/8 calls
gate:   pass
```

Empty findings → no body claims → has_sources fails → escalate. Never ship an uncited brief.

Budget. Soft warns. Hard raises

Live: `max_usd=0.20`, `max_calls=8`, `soft_usd=0.10`. Perplexity costs 0.006 per call. Fixture costs nothing, which is why Saturday still teaches the cap.

The ninth search does not run. `BudgetExceeded` is a `RuntimeError`, not a nudge.

Four stops: call budget, dollar budget, stable failure, no-source escalates.

Spillwave Solutions | spillwave.com

Troubleshooting

SYMPTOM	CAUSE	FIX
Reaching for a model in <code>check_brief</code>	stall	<code>return brief.check(...)</code>
Over-designed planner	stall	three strings
<code>task loop:research</code> missing	engine deleted	<code>sol3_research_deep_agents/loop.py</code>
Uncited brief shipped	skipped <code>has_sources</code>	empty findings escalate
Em dashes in the brief	model argued	<code>brief.strip_em_dashes</code>

Fall behind

```
cp loop.py loop.py.my-attempt
```

Watch Rick type. There is no drop-in. Later: `git checkout -- loop.py`.

Take-home ports are a different runtime, not a drop-in for this stub.

Spillwave Solutions | spillwave.com

Recap

Same graph as Module 1. New object: a question. New wall: one search tool.

Cost is an architecture problem, not a pricing problem. Do not shop for a cheaper model first.

Takeaways

1. Three sub-questions is a plan you can check.
2. Citations are arithmetic.
3. The orchestrator sees a summary, never the dump.
4. Merge is not a tool this loop holds.
5. Saturday object is a cited brief. Paper pipelines are take-home.